

西南科技大学计算机科学与技术学院

实验报告

实 验 名 称 开源项目开发管理

实 验 地 点 东 11c225

实 验 日 期 2024 年 11 月 29 日

指 导 教 师 韦勇

学 生 班 级

学 生 姓 名

学 生 学 号

提 交 日 期

一、实验目的

任务一：

掌握开放源代码项目的基本开发过程，熟练使用源代码版本管理网站，能参与开放源代码项目。

任务二：

1) 在 gitlab 中注册帐号，查找 linuxcourse/linuxer 项目（地址：<http://192.168.217.240/explore>），每组组长申请访问权限（通过访问 linuxcourse/linuxer 项目的 Request Access（申请权限）功能，如下图：）



2) 每个组创建至少 3 个 issues（地址：<http://192.168.217.240/linuxcourse/linuxer/-/issues>），标题为“xx 时间 xx 小组：改进点名称”（时间是对分易中的上课时间段，如周二晚；小组编号见对分易中的分组列表，如组 7），在描述信息中注明小组成员的姓名和学号，设置 issues 的 Milestone 为“实验 8 开源项目开发管理”，需要使用 issues 的讨论和交流功能进行讨论，至少有 3 条讨论记录。

二、实验设计

任务一：

登录你的帐号，新建项目，重现一下 7 实验指导中的步骤

任务二：

- 1、申请权限
- 2、创建 issues

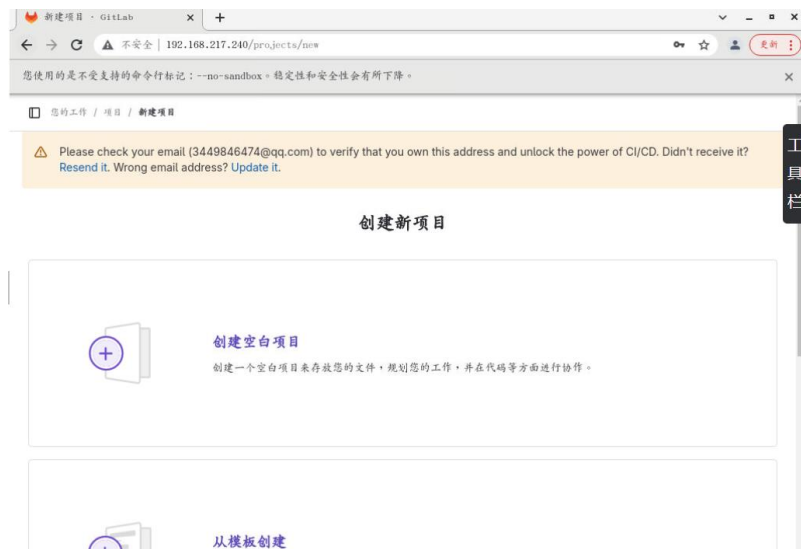
三、实验记录

任务一：

1. 首先创建好 gitlab 的账号，然后再登录进去



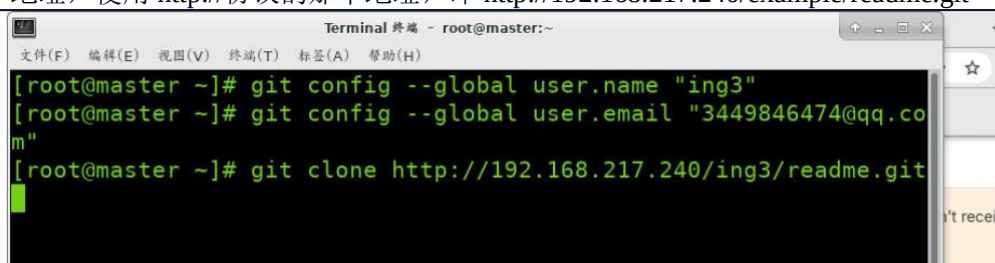
2. 登录进去后，创建一个新项目，点击创建空白项目

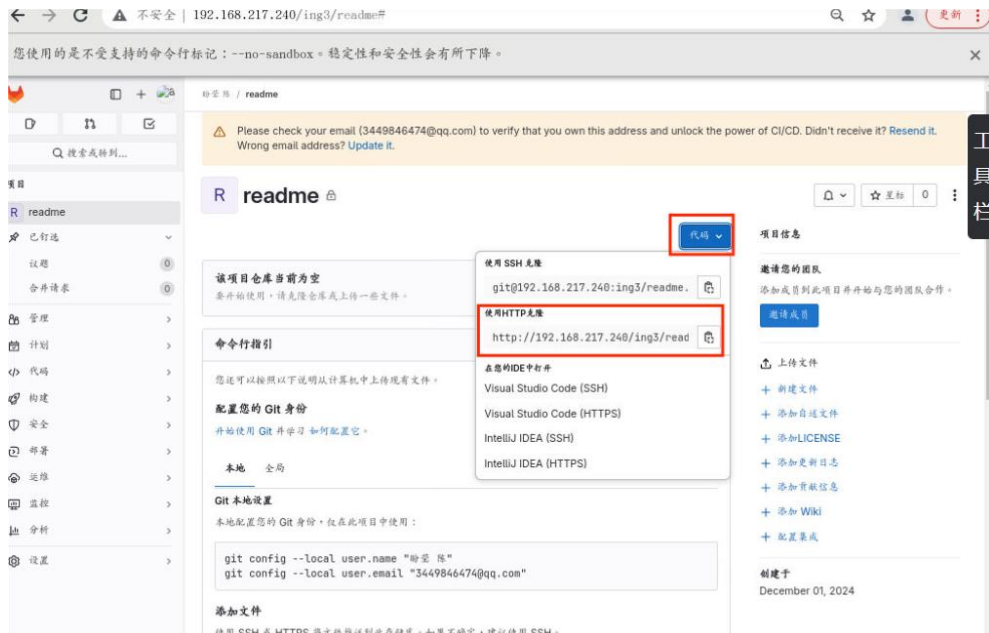


3. 为刚刚创建好的空白项目命名为 `readme`，同时取消勾选使用自述文件初始化仓库



4. 在终端中进行配置，设置 Git 全局身份，输入用户名和邮箱，再找到"clone"按钮下的代码库地址，使用 `http://`协议的那个地址，即 `http://192.168.217.240/example/readme.git`





`git config --global user.name "ing3"`: 这个命令设置了全局的 Git 用户名为"ing3"。这意味着在您的系统上,所有新的 Git 仓库都将使用这个用户名,除非在特定仓库中进行了不同的设置。

`git config --global user.email "example@example.com"`: 这个命令设置了全局的 Git 用户邮箱为"example@example.com"。这将用于提交时记录作者信息。

`git clone http://192.168.217.240/example/readme.git`: 这个命令尝试从指定的 HTTP URL 克隆一个名为"readme.git"的 Git 仓库。

由于我们设置仓库为私有,则登录时即需要输入仓库的用户名和密码

5. 输入用户名和密码登录,就可以将在网站上创建的 Git 仓库克隆到本地计算机上,在本地环境中使用 Git 的强大功能

```
[root@master ~]# git clone http://192.168.217.240/ing3/readme.git
正克隆到 'readme'...
warning: 远程 HEAD 指向一个不存在的引用,无法检出。
```

6. 登录成功后再次查看根目录下的文件,发现多出了一个 readme 文件, cd 进该文件中,然后创建 README.md 文件,再将 README.md 文件添加到 Git 的暂存区(暂存区是一个准备提交的更改的区域)

`git commit -m "add README"`: 这个命令将暂存区中的更改提交到本地仓库。-m 参数后面跟着的是提交信息,这里是 "add README",用于描述这次提交的目的。

`git push -u origin master`: 这个命令将本地的 master 分支推送到远程仓库 origin。-u 参数是用来设置上游(即远程跟踪分支),这样在以后推送时就可以省略远程仓库和分支的名称。

```

[root@master ~]# ll
total 0
drwxr-xr-x  1 root root  34 10月  9  2021 Desktop
drwx----- 1 root root  61 10月  9  2021 Downloads
drwxr-xr-x  1 root root 167 10月  9  2021 experiment
drwxr-xr-x  1 root root  30 9月  24  2021 IdeaProjects
drwxrwxr-x. 12 1000 1000 201 3月  24  2016 nltk_data
drwxr-xr-x  3 root root  30 9月  1  2021 PycharmProjects
drwxr-xr-x  3 root root  26 12月  1 20:14 readme
drwxr-xr-x  1 root root  52 10月 11  2021 tools
drwxr-xr-x  1 root root  39 11月 20  2021 workspace
[root@master ~]# cd readme
[root@master readme]# touch README
[root@master readme]# touch README.md
[root@master readme]# git add README.md
[root@master readme]# git commit -m "add README"
[master (根提交) 7e64803] add README
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 README.md

```

这时第一次推送到远程仓库，所以需要输入远程仓库的用户名和密码进行身份验证

```

drwxr-xr-x  1 root root  30 9月  24  2021 IdeaProjects
drwxrwxr-x. 12 1000 1000 201 3月  24  2016 nltk_data
drwxr-xr-x  3 root root  30 9月  1  2021 PycharmProjects
drwxr-xr-x  3 root root  26 12月  1 20:14 readme
drwxr-xr-x  1 root root  52 10月 11  2021 tools
drwxr-xr-x  1 root root  39 11月 20  2021 workspace
[root@master ~]# cd readme
[root@master readme]# touch README
[root@master readme]# touch README.md
[root@master readme]# git add README.md
[root@master readme]# git commit -m "add README"
[master (根提交) 7e64803] add README
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 README.md
[root@master readme]# git push -u origin master

```



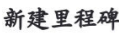
The dialog box is titled "OpenSSH". It contains a question mark icon and the text "Password for 'http://cpy12345@192.168.217.240':". Below this is a password input field with a single character visible. At the bottom, there is a button labeled "取消(C)" (Cancel) and a button labeled "确定(O)" (OK). A message "Passphrase length hidden intentionally" is displayed above the buttons.

```

create mode 100644 README.md
[root@master readme]# git push -u origin master
Counting objects: 3, done.
Writing objects: 100% (3/3), 209 bytes | 0 bytes/s, done.
Total 3 (delta 0), reused 0 (delta 0)
To http://192.168.217.240/ing3/readme.git
* [new branch]      master -> master
分支 master 设置为跟踪来自 origin 的远程分支 master。
[root@master readme]#

```

7. 在 readme 项目下创建两个里程碑 “write Chapter1”，“write Chapter2”



标题

write Chapter1

开始日期

2024-12-03
[清除开始日期](#)

截止日期

2024-12-10
[清除截止日期](#)

描述

预览 | B I S | | = < > ↺ ⋮ ≡ ↶ ⌂ ✎

写入里程碑描述...

ling3 / readme / 里程碑

开放中 2

已关闭 0

全部 2

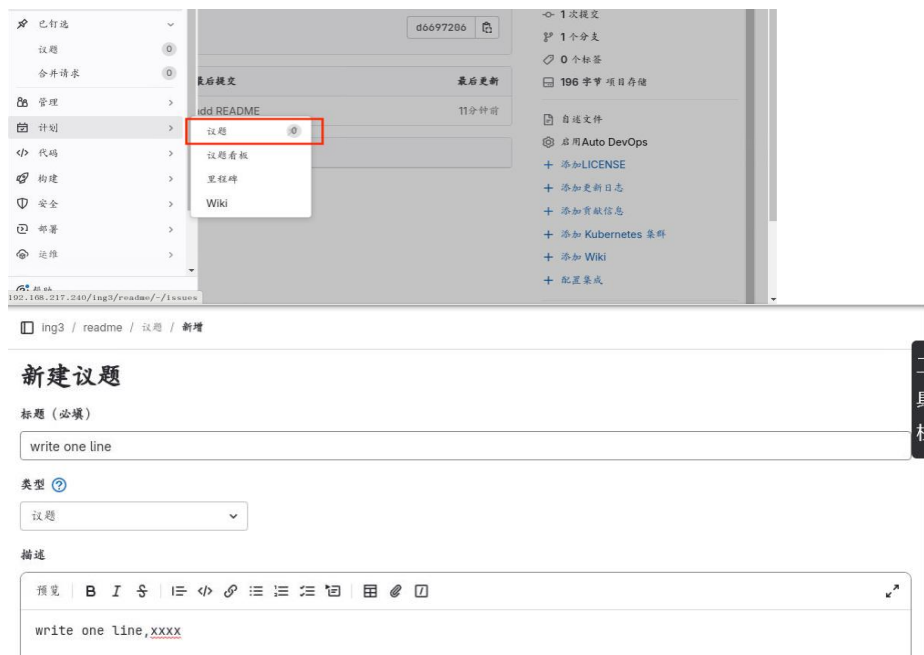
按里程碑名称筛选

即将截止 ▾

新建里程碑

write Chapter2	ling3 / readme	12月3, 2024 - 12月10, 2024	开放中	0/0 完成 0%	
write Chapter1	ling3 / readme	12月3, 2024 - 12月10, 2024	开放中	0/0 完成 0%	

8. 创建 issue，填写标题，指派人，截止日期，里程碑



指派人
ing3

截止日期
2024-12-09

里程碑
write Chapter1

标记
选择标记

创建 议题 取消

9. 创建 Merge Request，下拉按钮创建合并请求，修改分支名称和源

write one line

开放中 议题 由 ing3 创建于 刚刚

write one line,xxxx

0 0

创建合并请求

创建合并请求及分支
创建分支

分支名称
1-write-chapter1-first

源(分支或标签)
master

创建合并请求

标记为已完成

1位指派人
ing3

标记
无

里程碑
write Chapter1

权重
此功能已锁定 进一步了解

截止日期
2024年12月9日 - 删除截止日期

工时统计
无预计或已用时间

然后分支就创建成功

ing3 / readme / 合并请求 / 11

Draft: Resolve "write one line"

开放中 ing3 请求将 1-write-one-line 合并到 master 2分钟后

概览 0 提交 0 流水线 0 变更 0

标记为已完成

1位指派人
ing3

10. 先从远程仓库（这里的远程仓库别名为 origin）获取最新的分支和提交信息，但不更新到本地中，然后创建并切换到一个新的名为 1-write-chapter1-first 的本地分支。

-b "1-write-chapter1-first": 创建一个的新分支

origin/1-write-chapter1-first: 指定新分支的起点是远程仓库 origin 中的 1-write-chapter1-first 分支

```
[root@master readme]# ll
total 0
-rw-r--r-- 1 root root 0 12月  3 20:42 README.md
[root@master readme]# git fetch origin
来自 http://192.168.217.240/ing3/readme
* [新分支]          1-write-chapter1-first -> origin/1-write-chapter1-first
* [新分支]          1-write-one-line -> origin/1-write-one-line
[root@master readme]# git checkout -b "1-write-chapter1-first" "origin/1-write-chapter1-first"
分支 1-write-chapter1-first 设置为跟踪来自 origin 的远程分支 1-write-chapter1-first。
切换到一个新分支 '1-write-chapter1-first'
[root@master readme]#
```

11. 在之前创建的 README.md 文档中写入一些字符，即在新分支 1 - write - chapter1 - first 下对项目文档进行更改，将已跟踪文件 README.md 的修改提交到本地仓库，并附上提交

信息"update README", 在新分支下记录对 README.md 文件的更新

```
Everything up-to-date
[root@master readme]# vi README.md
[root@master readme]# git commit -a -m "update README"
```

12. 将本地分支 1 - write - chapter1 - first 的提交推送到远程仓库 (origin)，即本地对 README.md 文件的修改推送到远程仓库的对应分支，执行这条命令后，终端显示了推送的详细信息，包括推送的对象数量、字节数等。并且给出了一个远程查看合并请求的链接：http://192.168.217.240/ing3/readme/-/merge_requests/2，这表明推送成功并且可以在远程仓库中查看合并请求。

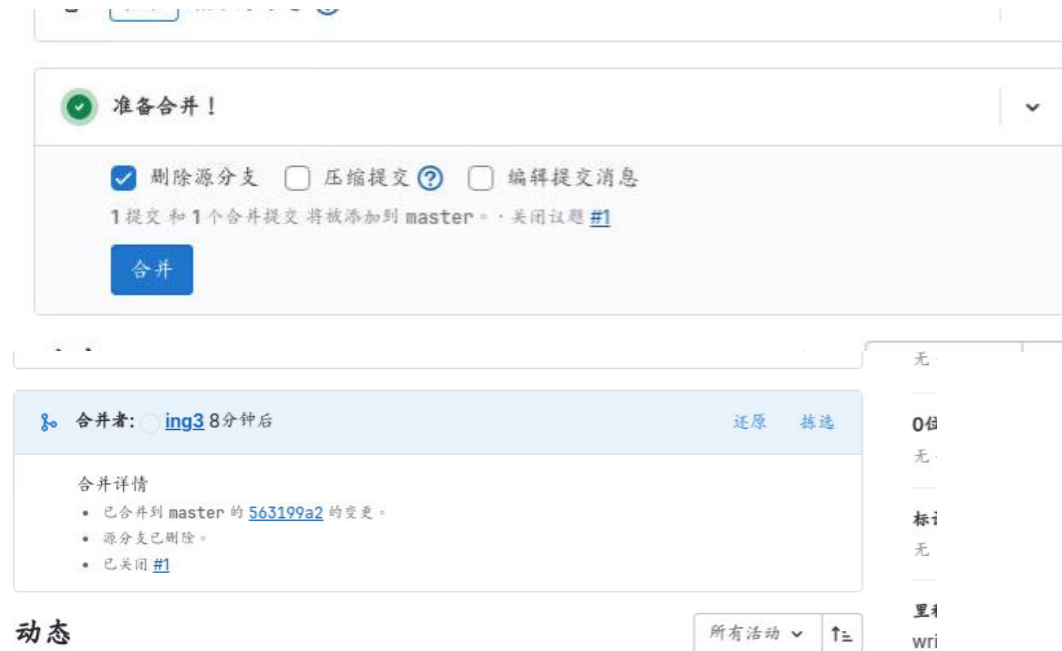
```
[root@master readme]# vi README.md
[root@master readme]# git commit -a -m "update README"
[1-write-chapter1-first 75f9f6e] update README
1 file changed, 1 insertion(+)
[root@master readme]# git push origin 1-write-chapter1-first
Counting objects: 5, done.
Writing objects: 100% (3/3), 247 bytes | 0 bytes/s, done.
Total 3 (delta 0), reused 0 (delta 0)
remote:
remote: View merge request for 1-write-chapter1-first:
remote: http://192.168.217.240/ing3/readme/-/merge_requests/2
remote:
To http://192.168.217.240/ing3/readme.git
    d669720..75f9f6e 1-write-chapter1-first -> 1-write-chapter1-first
[root@master readme]# git push origin "1-write-chapter1-first"
Everything up-to-date
```

13. 在 GitLab 中可以看到分支 1-write-chapter1-first 下更新了一个提交

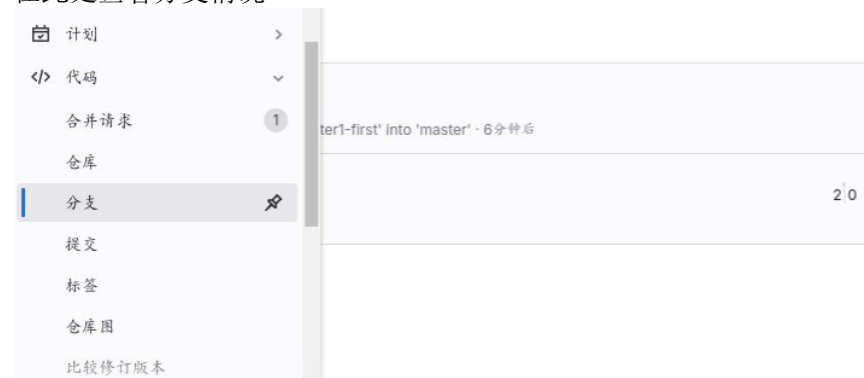


14. 然后在概览中可以看到刚刚更新 README 文档的记录，然后再点击合并请求的按钮，再点击开始合并，将分支 1-write-chapter1-firs 合并到 master 下，然后就合并成功了，之后就会删除原分支





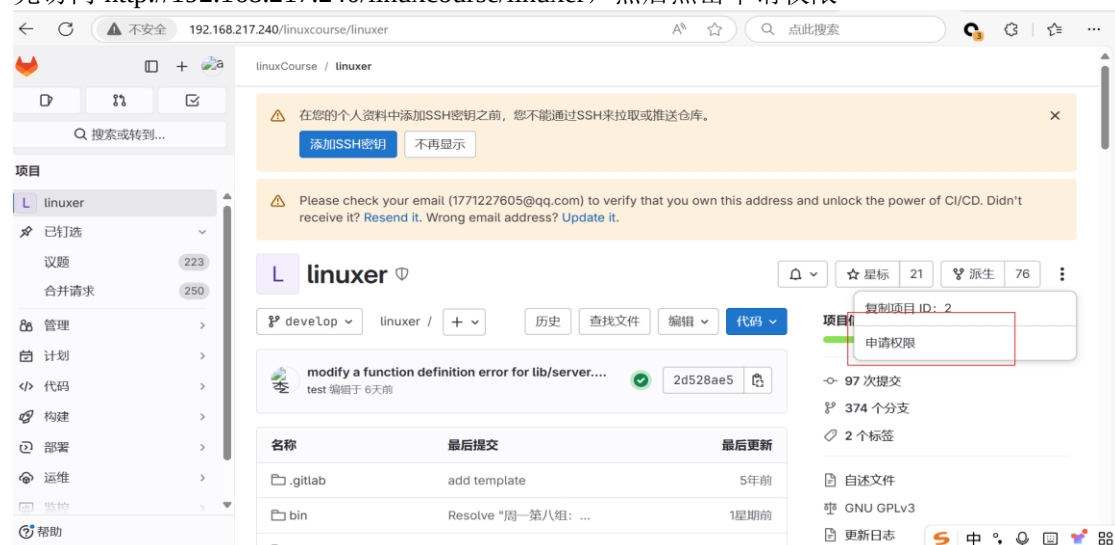
在此处查看分支情况



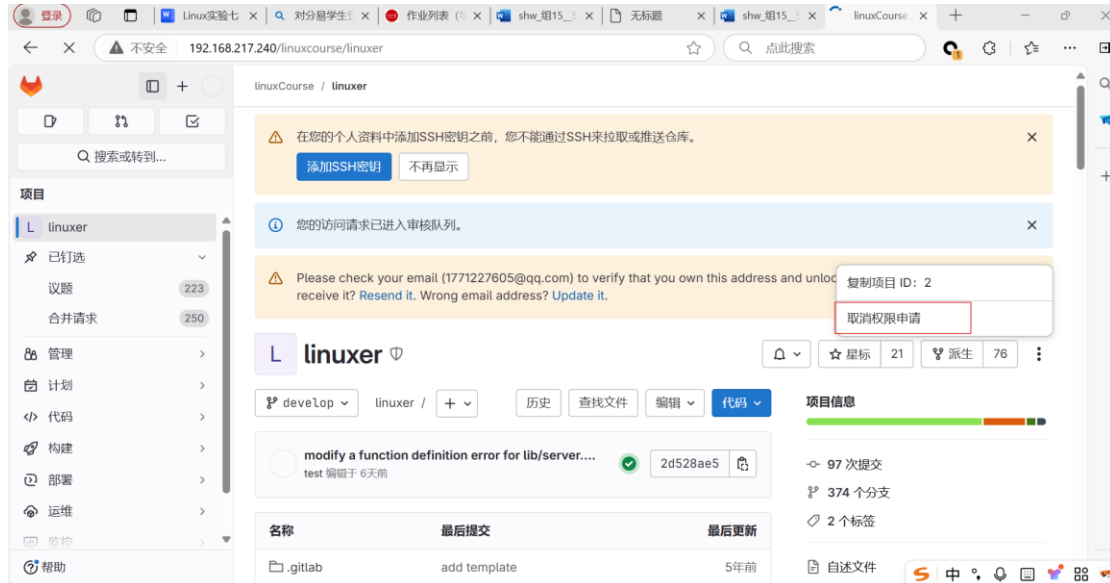
任务二:

1) 在 gitlab 中注册帐号, 查找 linuxcourse/linuxer 项目(地址: <http://192.168.217.240/explore>), 每组组长申请访问权限(通过访问 linuxcourse/linuxer 项目的 Request Access (申请权限) 功能):

先访问 <http://192.168.217.240/linuxcourse/linuxer>, 然后点击申请权限



成功申请权限



2) 每个组创建至少 3 个 issues（地址：<http://192.168.217.240/linuxcourse/linuxer/-/issues>），标题为“xx 时间 xx 小组：改进点名称”（时间是对分易中的上课时间段，如周二晚；小组编号见对分易中的分组列表，如组 7），在描述信息中注明小组成员的姓名和学号，设置 issues 的 Milestone 为“实验 8 开源项目开发管理”，需要使用 issues 的讨论和交流功能进行讨论，至少有 3 条讨论记录：

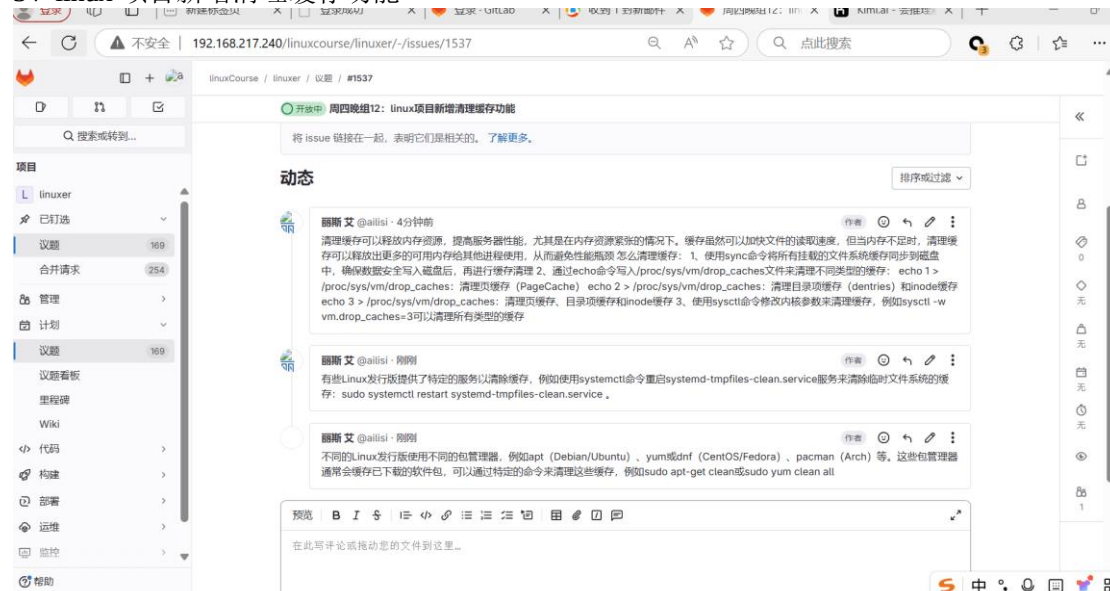
1、添加社交功能



2、提高 linux 项目用户安全性



3、linux 项目新增清理缓存功能



四、实验思考或体会

任务一：

解释 GitLab 提供的持续集成（CI）功能。（提示：可以查看实验服务器 <http://192.168.217.240/> 中的帮助以及 linuxer 项目的 CI 功能实现）

持续集成是一种软件开发实践，它要求开发团队成员频繁地将代码集成到共享的代码库中。每次集成都会通过自动化构建、测试等流程来快速检测代码的合并是否引入了问题，以便尽早发现并解决错误，确保软件质量的稳定以及加快软件的交付周期。GitLab 的 CI 功能就是为了帮助开发者方便地在项目中实现这样的持续集成流程。

在 GitLab 中，通过 `.gitlab-ci.yml` 文件来定义持续集成的相关配置，它存放在项目仓库的根目录下。linuxer 项目的 CI 功能实现展示了其基本的配置结构和可用的一些配置选项：

//预先定义在后续构建和部署等阶段会用到的变量

variables:

IMAGE_NAME: "linuxer:latest" //指定构建出来的镜像名称以及版本等相关用途。

DOCKER_FILE_PATH: "./Dockerfile" //明确 Dockerfile 所在的路径

//定义了整个持续集成过程会经历的不同阶段顺序

stages:

- build //构建阶段
- deploy //部署阶段

build:

stage: build

tags: //指定执行此作业的特定运行器

- build

script:

//定义了实际要执行的命令脚本

- echo "start make image" //打印出 "start make image"

- make image_server //调用 make image_server 命令

deploy:

stage: deploy

tags:

- deploy

script:

- echo "docker run..." //打印 "docker run..."

- make start_server //调用 make start_server 命令

按照上述配置，当有代码推送到 GitLab 仓库触发 CI 流程时：

- 首先会进入 build 阶段，找到对应的带有 build 标签的运行器，在该运行器上执行 build 作业里定义的脚本，也就是先输出提示信息，然后尝试构建镜像（通过 make image_server 命令）。如果构建过程中出现错误，整个 CI 流程会在此阶段停止并反馈错误信息，便于开发者及时修复构建相关的问题。
- 若 build 阶段成功完成，接着就会进入 deploy 阶段，GitLab 会寻找带有 deploy 标签的运行器来执行 deploy 作业。执行脚本中先是输出提示信息，然后尝试启动相关服务（通过 make start_server 命令），从而完成从代码更新到构建镜像再到部署启动服务这样一个基本的持续集成及简单部署的流程。